v0.45 Security Implementation Report

Review date: 11 November 2026 **Baseline:** v0.44

Release verdict: NO-GO. Source-level improvements were tested offline, but the full live security and deployment gate remains unverified. This document does not certify the platform as hacker-proof.

Executive summary

Version 0.45 addresses a specific multi-server login-abuse weakness. Login requests now reserve an account attempt in PostgreSQL before the Argon2id password check. A shared 15-minute window permits 10 attempts; additional attempts are denied with HTTP 429. The database stores a keyed HMAC digest, not the email address or password.

If MFA is required, a valid password alone does not clear the attempt window. The counter clears only after successful MFA. Existing single-use MFA challenges and per-API throttling remain additional layers. The new signing key is neither generated in source nor sent to the browser.

Implementation inventory

Control	Implementation and boundary
Distributed login throttling	Atomic PostgreSQL INSERT ... ON CONFLICT reservation before password checking.
Secret-safe counters	HMAC-SHA256 keyed on normalized email; no plaintext account identifier, IP address, password, or token stored in this table.
MFA-safe state	Failed or pending MFA does not clear the counter; successful MFA does.
Fail-closed configuration	Production startup rejects missing, invalid or reused LOGIN_ABUSE_PEPPER.
Schema and CI	New forward-only SQL migration 020, manifest fingerprint, CI secret generation, offline tests and guarded PostgreSQL concurrency harness.

Scope and limitations

This checkpoint does not turn on database row-level security across tenant data. Existing pooled queries do not consistently carry transaction-scoped tenant context. Retrofitting RLS without a database-role and query migration would risk outages or privilege bypass. A separate adversarially tested RLS rollout is required.

ENGINEERING EVIDENCE

Verification and unresolved gates

Check	Observed result
New v0.45 unit/service/controller contract tests	11/11 PASS
Historical and current version suites (v0.03-v0.45)	43/43 PASS
All offline checks including architecture, syntax, CI, scan	47/47 PASS
TS/TSX syntax transpilation	100 files, 0 diagnostics; not a full typecheck
SQL migration checksums	20/20 matched source manifest
High-confidence snapshot secret scan	0 pattern matches across 1,619 files at time of run; does not cover Git history
Full API/frontend build	FAILED: npm dependencies and Node types not installed
Live database concurrency and two-server behavior	NOT RUN; guarded runner requires approved disposable PostgreSQL
History-inclusive secrets scan, dependency audit, TLS and browser testing	NOT VERIFIED

Priority risk register

Severity	Finding	Next action
HIGH	Cross-server login throttling lacks live evidence	Run concurrent reservations on disposable PostgreSQL and two real API servers.
HIGH	Targeted account lockout may cause denial of service	Add edge/IP controls, alerts, support recovery and operational tests.
HIGH	Database tenant-level RLS absent	Introduce least-privilege DB role, transaction-scoped tenant context, RLS and negative cross-tenant tests.
HIGH	Dependency and history secrets audit not complete	Create authentic lockfile, npm ci, audit, full Git scan, and rotate any real exposed credentials.
HIGH	Evidence uploads not verified for malware	Quarantine and scan permitted files; test bypasses and access restrictions.

RELEASE DECISION AND NEXT STEPS

Production release remains blocked

Offline tests are useful evidence for deterministic policy behavior, but they do not prove real database isolation, TLS, secrets rotation, cloud perimeter controls, strong dependency hygiene, correct browser handling or a successful recovery from compromise.

Required completion sequence

1. Trusted dependency graph. Generate and review a real package-lock.json, run npm ci, typecheck, API/web builds, npm audit, SBOM and remediation.

2. PostgreSQL integration. Back up an approved disposable DB; apply migration 020; run multi-connection contention, login/MFA replay and row-lock checks.

3. Identity and tenant security. Run two API instances; implement and test database-enforced RLS with least privilege, cross-tenant denial, and no bypass via privileged DB roles.

4. Perimeter and content security. Verify HTTPS/HSTS, secure cookies/CSRF, SSH/firewall exposure, malware scan, admin MFA, price enforcement, logs and backup restores.

5. Repository and release governance. Scan complete canonical Git history, rotate confirmed exposed secrets, perform adversarial testing and retain signed-off evidence.

Operator files

See V45_KALI_VALIDATION.md for guarded commands, TEST_EVIDENCE_V45.md for executed results, V45_HANDOVER.md for changed modules, and SECURITY_FINALIZATION_STANDARD.md for the permanent cross-project security gate.

Final verdict: NO-GO. Do not host real land ownership evidence or accept paying customers until all required release gates pass and any critical/high findings are resolved.